Human Verification: `TrajectoryComposition`

Composing Trajectory Evaluation with Trust Evaluation

Machine-generated from `lean/TrajectoryComposition.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `TrajectoryComposition` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean.

The module defines a grade-level map ψ : `TLevel` $\to$ `TrustLevel` from the five-level trajectory chain to the seven-level deployed trust chain, verifies that ψ is a chain morphism (monotone, bottom-preserving), and applies the lax transport theorem (`tau_transport` from `Trust.lean`) to concrete trajectories.

Contents

1 Overview **1**

2 The trust chain **2**
2.1 Inductive type: `TrustLevel` (local copy) 2
2.2 Definition: `TrustLevel.le` and `Chain TrustLevel` 2

3 The trajectory-trust morphism **3**
3.1 Definition: `psi` 3
3.2 Theorem: `psi_bot` 3
3.3 Theorem: `psi_mono` 3

4 Concrete transport results **4**
4.1 Theorem: `psi_circularTraj_bot` 4
4.2 Theorem: `psi_honest_e4` 4
4.3 Theorem: `psi_honest_e3` 4

5 Translation conventions **5**

1 Overview

The module bridges two trust chains. On one side is the five-level trajectory chain `TLevel` (defined in `Trajectory.lean`): `unsupported` $<$ `shallow` $<$ `independent` $<$ `crossVerified` $<$ `mechChecked`. On the other is the seven-level deployed agent chain `TrustLevel` used in production registries: `unchecked` $<$ `speculative` $<$ `computed` $<$ `proved` $<$ `peerReviewed` $<$ `published` $<$ `leanVerified`.

The map ψ translates: `unsupported` $\mapsto$ `unchecked`, `shallow` $\mapsto$ `computed`, `independent` $\mapsto$ `proved`, `crossVerified` $\mapsto$ `peerReviewed`, `mechChecked` $\mapsto$ `leanVerified`.

2 The trust chain

2.1 Inductive type: `TrustLevel` (local copy)

Definition 2.1 (Deployed trust levels)**.** A seven-element chain mirroring `TrustBoundary.TrustLevel`:

`unchecked` $<$ `speculative` $<$ `computed` $<$ `proved` $<$ `peerReviewed` $<$ `published` $<$ `leanVerified`.

Lean.

```
inductive TrustLevel where
  | unchecked
  | speculative
  | computed
  | proved
  | peerReviewed
  | published
  | leanVerified
deriving DecidableEq, Repr
```

Haskell.

```
data TrustLevel
  = Unchecked | Speculative | Computed | Proved
  | PeerReviewed | Published | LeanVerified
  deriving (Eq, Ord, Show)
```

Fidelity note. This type is defined locally in `Trajectory` namespace, mirroring the one in `TrustBoundary.lean`. The `Chain TrustLevel` instance here is also proved locally.

2.2 Definition: `TrustLevel.le` and `Chain TrustLevel`

Definition 2.2 (Order on `TrustLevel`)**.** The order is defined case-by-case, establishing the chain `unchecked` $\leq \cdots \leq$ `leanVerified`. The `Chain TrustLevel` instance has $\bot =$ `unchecked`.

Lean.

```
def TrustLevel.le : TrustLevel → TrustLevel → Prop
  | .unchecked, _ ⇒ True
  | .speculative, .unchecked ⇒ False
  | .speculative, _ ⇒ True
  -- ... (remaining cases follow the chain order)
  | .leanVerified, .leanVerified ⇒ True
  | .leanVerified, _ ⇒ False

instance : Chain TrustLevel where
  bot := .unchecked
  -- reflexivity, transitivity, antisymmetry, totality, bot_le, dec
     all proved
```

Haskell.

```
-- Derived Ord suffices; bot = Unchecked.
```

3 The trajectory-trust morphism

3.1 Definition: `psi`

Definition 3.1 (Grade translation map)**.** The function $\psi : \texttt{TLevel} \to \texttt{TrustLevel}$ translates trajectory grades to deployed trust grades:

$$\begin{aligned} \psi(\texttt{unsupported}) &= \texttt{unchecked}, \\ \psi(\texttt{shallow}) &= \texttt{computed}, \\ \psi(\texttt{independent}) &= \texttt{proved}, \\ \psi(\texttt{crossVerified}) &= \texttt{peerReviewed}, \\ \psi(\texttt{mechChecked}) &= \texttt{leanVerified}. \end{aligned}$$

Lean.

```
def psi : TLevel → TrustLevel
  | .unsupported  ⇒ .unchecked
  | .shallow      ⇒ .computed
  | .independent  ⇒ .proved
  | .crossVerified ⇒ .peerReviewed
  | .mechChecked  ⇒ .leanVerified
```

Haskell.

```
psi :: TLevel → TrustLevel
psi Unsupported  = Unchecked
psi Shallow      = Computed
psi Independent  = Proved
psi CrossVerified = PeerReviewed
psi MechChecked  = LeanVerified
```

3.2 Theorem: `psi_bot`

Theorem 3.2 (ψ preserves bottom)**.**

$$\psi(\bot_{\texttt{TLevel}}) = \bot_{\texttt{TrustLevel}}, \quad \text{i.e.,} \quad \psi(\texttt{unsupported}) = \texttt{unchecked}.$$

This is the strictness-at-bottom condition required for ψ to be a valid trust-system morphism (see `Mor.phi_bot` in `Trust.lean`).

Lean.

```
theorem psi_bot :
    psi (Chain.bot : TLevel) = (Chain.bot : TrustLevel)
```

Haskell.

```
-- psi Unsupported == Unchecked   -- True by definition
```

3.3 Theorem: `psi_mono`

Theorem 3.3 (ψ is monotone)**.** For all $a, b : \texttt{TLevel}$,

$$a \le b \implies \psi(a) \le \psi(b).$$

Together with `psi_bot`, this makes ψ a valid `phi` component of a `Mor` in the sense of `Trust.lean`.

Lean.

```
theorem psi_mono : ∀ {a b : TLevel}, a ≤ b → psi a ≤ psi b
```

Haskell.

```haskell
-- psi is order-preserving: a <= b ⇒ psi a <= psi b
-- Follows from the chain structure and the explicit case table of psi
    .
```

4 Concrete transport results

4.1 Theorem: `psi_circularTraj_bot`

Theorem 4.1 (Circular trajectory maps to unchecked under ψ)**.** For every event e : `Evt`,

$$\psi(\tau(\texttt{circularTraj}, e)) = \texttt{unchecked}.$$

This follows immediately from `circularTraj_tau_bot` and `psi_bot`.

Lean.

```lean
theorem psi_circularTraj_bot : ∀ (e : Evt),
    psi (tau circularTraj circularTraj_wf e) = TrustLevel.unchecked
```

Haskell.

```haskell
-- psi (tau circularTraj e) == Unchecked  for all e
```

4.2 Theorem: `psi_honest_e4`

Theorem 4.2 (Honest trajectory's verify event maps to proved)**.**

$$\psi(\tau(\texttt{honestTraj}, e_4)) = \texttt{proved}.$$

Recall from `Trajectory.lean` that $\tau(\texttt{honestTraj}, e_4) = \texttt{independent}$, and $\psi(\texttt{independent}) = \texttt{proved}$.

Lean.

```lean
theorem psi_honest_e4 :
    psi (tau honestTraj honestTraj_wf Evt.e4) = TrustLevel.proved
```

Haskell.

```haskell
-- psi (tau honestTraj E4) == Proved
```

4.3 Theorem: `psi_honest_e3`

Theorem 4.3 (Honest trajectory's claim event maps to computed)**.**

$$\psi(\tau(\texttt{honestTraj}, e_3)) = \texttt{computed}.$$

Recall that $\tau(\texttt{honestTraj}, e_3) = \texttt{shallow}$, and $\psi(\texttt{shallow}) = \texttt{computed}$.

Lean.

```lean
theorem psi_honest_e3 :
    psi (tau honestTraj honestTraj_wf Evt.e3) = TrustLevel.computed
```

Haskell.

```haskell
-- psi (tau honestTraj E3) == Computed
```

5 Translation conventions

1. **Proof bodies omitted.** Lean machine-checks all proofs.
2. **Local type duplication.** `TrustLevel` is defined independently here (mirroring `TrustBoundary.TrustLe` to keep the file self-contained and avoid namespace friction. A future refactor could import from `TrustBoundary.lean` instead.
3. **The morphism condition.** `psi_bot` and `psi_mono` are exactly the `phi_bot` and `phi_mono` fields required to extend ψ to a `Mor` between trust systems. The transport theorem (`tau_transport`) then applies: importing along such a morphism never claims more than ψ of the source's canonical trust.
4. **Concrete transport.** Rather than constructing a full `Mor` record (which would require matching step-sets and premise maps between the trajectory and trust systems), the module applies results from `Trajectory.lean` directly, composing with ψ pointwise.